

Function in Python

```
In [2]: def hello():  
        print('good evening')
```

```
In [3]: def hello():  
        print('good evening')  
        hello()
```

good evening

```
In [4]: def hello():  
        print('good evening')  
        hello()  
  
        def hello():  
            print('good evening')  
            hello()  
  
        def hello():  
            print('good evening')  
            hello()
```

good evening
good evening
good evening

```
In [5]: def hello():  
        print('good evening')  
  
        hello()  
  
        hello()  
  
        hello()  
  
        hello()
```

good evening
good evening
good evening
good evening

```
In [6]: def add(x,y):  
        c=x+y  
        print(c)  
  
        add(5,6)
```

11

```
def add(x,y,z): c = x + y + z print(c)
```

```
add(5,6)
```

value for z is missing. Therefore, system is throwing below error message

```
In [8]: def add(x,y,z):  
        c = x + y + z  
        print(c)  
  
        add(5,6,7)
```

18

```
In [9]: def add(x,y):  
        c=x+y  
        return c  
  
        add(5,6)
```

Out[9]: 11

Defining and calling multiple functions in the same cell works, but separating them improves readability and debugging.

```
def hello(): print('good evening') hello()
```

```
def add(x,y): c=x+y return c add(5,6)
```

```
In [12]: def hello():  
        print('good evening')  
  
        def add(x,y):  
            c=x+y  
            return c  
  
        hello()  
        add(5,6)
```

good evening

Out[12]: 11

```
In [13]: def greet():  
        print('hello')  
        print('good noon')  
  
        def add(x,y):  
            c=x+y  
            print(c)  
  
        def sub(x,y,z):  
            d = x-y-z
```

```
print(d)

greet()
add(5,4)
sub(10,2,4)
```

```
hello
good noon
9
4
```

The return statement sends those values back to the caller.

In Python, you can return multiple values by separating them with commas → Python actually packs them into a tuple.

```
def add_sub(x,y): c= x+y d= x-y return c, d

add_sub(4,5)
```

```
In [15]: def add_sub(x,y):
          c= x+y
          d= x-y
          return c, d

          result = add_sub(4,5)
          print(result)
          print(type(result))
```

```
(9, -1)
<class 'tuple'>
```

```
In [16]: def add_sub(x,y):
          c= x+y
          d= x-y
          return c, d

          result, result1 = add_sub(4,5)

          print(result)
          print(result1)

          print(type(result))
          print(type(result1))
```

```
9
-1
<class 'int'>
<class 'int'>
```

```
In [17]: def add_sub_mul(x,y):
          c= x+y
          d= x-y
          e = x*y
          return c, d, e

          result, result1, result2 = add_sub_mul(4,5)

          print(result)
          print(result1)
          print(result2)

          print(type(result))
          print(type(result1))
          print(type(result2))

9
-1
20
<class 'int'>
<class 'int'>
<class 'int'>
```

```
In [18]: def update():
          x = 8
          print(x)

          update()
```

8

```
In [20]: def add(a,b): # a & b are called as formal argument
          c = a+b
          print(c)

          add(5,6) # 5 & 6 is called actual argument
```

11

positional argument

```
def person(name,age): print(name) print(age)
```

```
person('nit', 22)
```

```
In [22]: def person(name,age):
          print(name)
          print(age)

          person(22, 'nit')
```

22
nit

```
In [23]: def person(name,age):
          print(name)
          print(age-1)

          person(22,'nit')
```

22

```
-----
TypeError                                Traceback (most recent call last)
Cell In[23], line 5
      2     print(name)
      3     print(age-1)
----> 5 person(22,'nit')

Cell In[23], line 3, in person(name, age)
      1 def person(name,age):
      2     print(name)
----> 3     print(age-1)

TypeError: unsupported operand type(s) for -: 'str' and 'int'
```

```
In [24]: def person(name,age):
          print(name)
          print(age+1)

          person('nit',22)
```

nit
23

```
In [25]: def person(name,age):
          print(name)
          print(age+1)

          person('nit')
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[25], line 5
      2     print(name)
      3     print(age+1)
----> 5 person('nit')

TypeError: person() missing 1 required positional argument: 'age'
```

```
In [26]: def person(name,age):
          print(name)
          print(age+1)

          person(22)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[26], line 5
      2     print(name)
      3     print(age+1)
----> 5 person(22)

TypeError: person() missing 1 required positional argument: 'age'
```

In [28]: `def person(name):`

```
    print(name)
    print(age+1)

person('nit', 22)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[28], line 6
      3     print(name)
      4     print(age+1)
----> 6 person('nit', 22)

TypeError: person() takes 1 positional argument but 2 were given
```

keyword argument

In [29]: `def person(name,age):`

```
    print(name)
    print(age+1)

person('nit', 22)
```

nit
23

In [30]: `def person(name, age):`

```
    print(name)
    print(age+1)

person(22, 'nit')
```

22

```

-----
TypeError                                Traceback (most recent call last)
Cell In[30], line 6
      3     print(name)
      4     print(age+1)
----> 6 person(22, 'nit')

Cell In[30], line 4, in person(name, age)
      1 def person(name, age):
      3     print(name)
----> 4     print(age+1)

TypeError: can only concatenate str (not "int") to str

```

```

In [31]: def person(name, age):

        print(name)
        print(age+1)

        person(age = 22, name = 'nit', phone = 879678)

```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[31], line 6
      3     print(name)
      4     print(age+1)
----> 6 person(age = 22, name = 'nit', phone = 879678)

TypeError: person() got an unexpected keyword argument 'phone'

```

```

In [32]: def person(name, age, phone):

        print(name)
        print(age+1)
        print(phone)

        person(age = 22, name = 'nit', phone = 879678)

```

```

nit
23
879678

```

```

In [33]: def person(name, age=18):

        print(name)
        print(age)

        person('nit')

```

```

nit
18

```

```

In [ ]: ## Check if a number is even or odd

```

```

In [36]: def even_or_odd(num):
        if num % 2 == 0:

```

```
        return "Even"
    else:
        return "Odd"

even_or_odd(7)
```

Out[36]: 'Odd'

```
In [37]: def even_or_odd(num):
        if num % 2 == 0:
            return "Even"
        else:
            return "Odd"

even_or_odd(50)
```

Out[37]: 'Even'

Reverse a string

```
In [40]: def reverse_string(text):
        return text[::-1]

reverse_string("NIT Technology")
```

Out[40]: 'ygonlohceT TIN'

In []: